

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Recurrence

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Admin 2
Review 4
Sequence Modeling 21
Composite Memory 29
Linear Recurrence 36
Scans 46
Output Modeling 53
Recurrent Loss Functions 58
Backpropagation Through Time 66
Recurrent Neural Networks 71
Coding 82

Admin

Admin

We had to cover basics (backpropagation, classification, etc)

Now, I think the lectures become more interesting

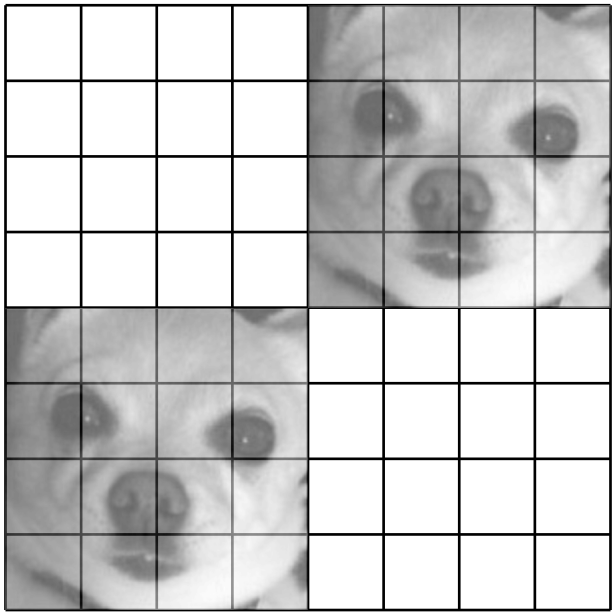
Focus on modern methods that solve hard problems

- Recurrent networks and long-term memory
- Autoencoders
- Probabilistic generative models
- Attention and transformers
- Imitation learning

Review

Review

In perceptrons, each neuron in a layer is independent



Review

We assume no relationship or ordering for input elements or parameter

$$\sigma(\theta_1 x_1 + \theta_2 x_2) \Rightarrow \sigma(\theta_2 x_2 + \theta_1 x_1)$$



These images are equivalent to a perceptron!

Review

A **signal** represents information as a function of time, space or some other variable

$$x(t) = 2t + 1$$

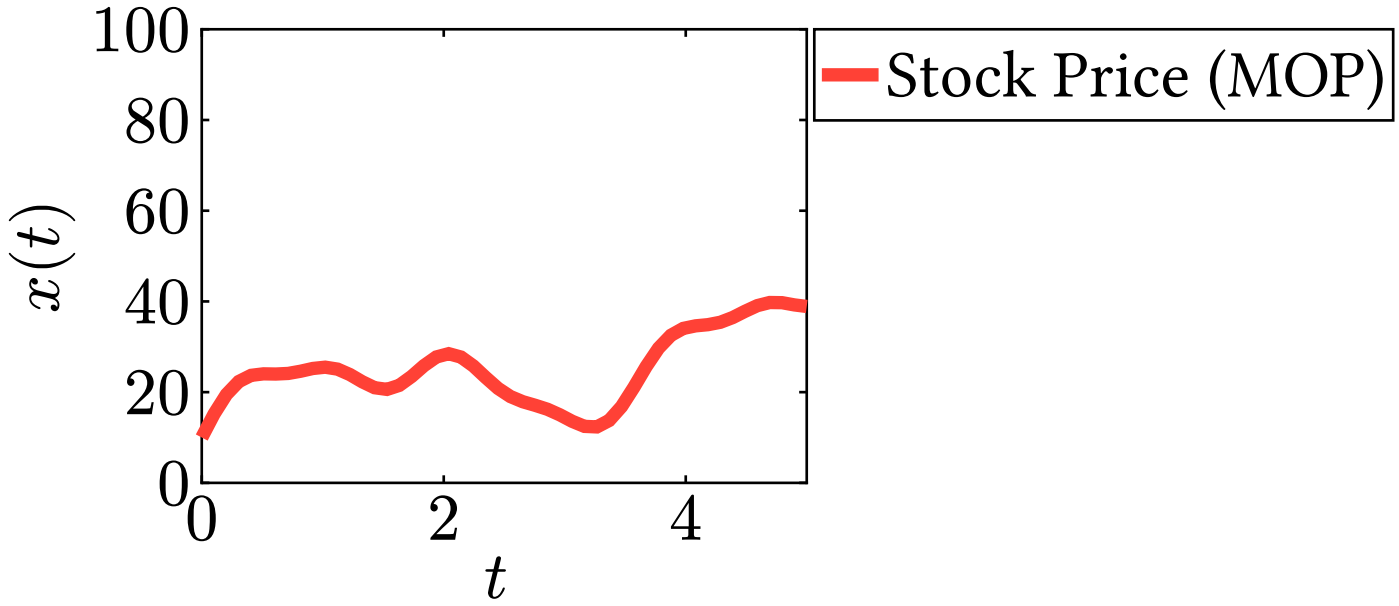
$$x(u, v) = \frac{u^2}{v} - 3$$

$x(t), x(u, v)$ represent physical processes that we may or may not know

In **signal processing**, we analyze the meaning of signals

Review

$$x(t) = \text{stock price}$$

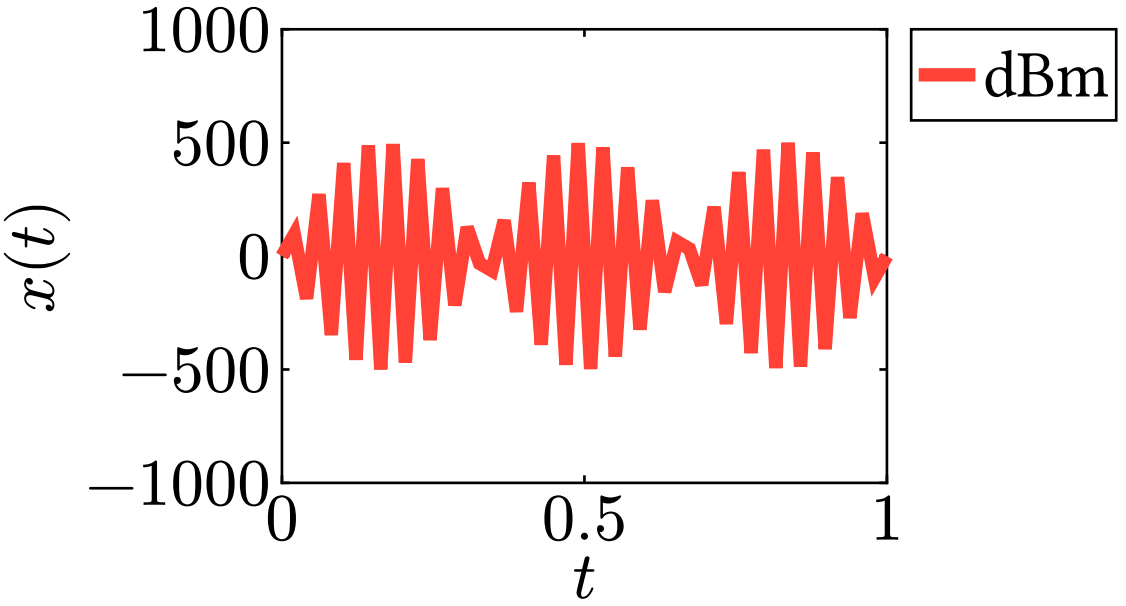


There is an underlying structure to $x(t)$

Structure: Tomorrow's stock price will be close to today's stock price

Review

$$x(t) = \text{audio}$$

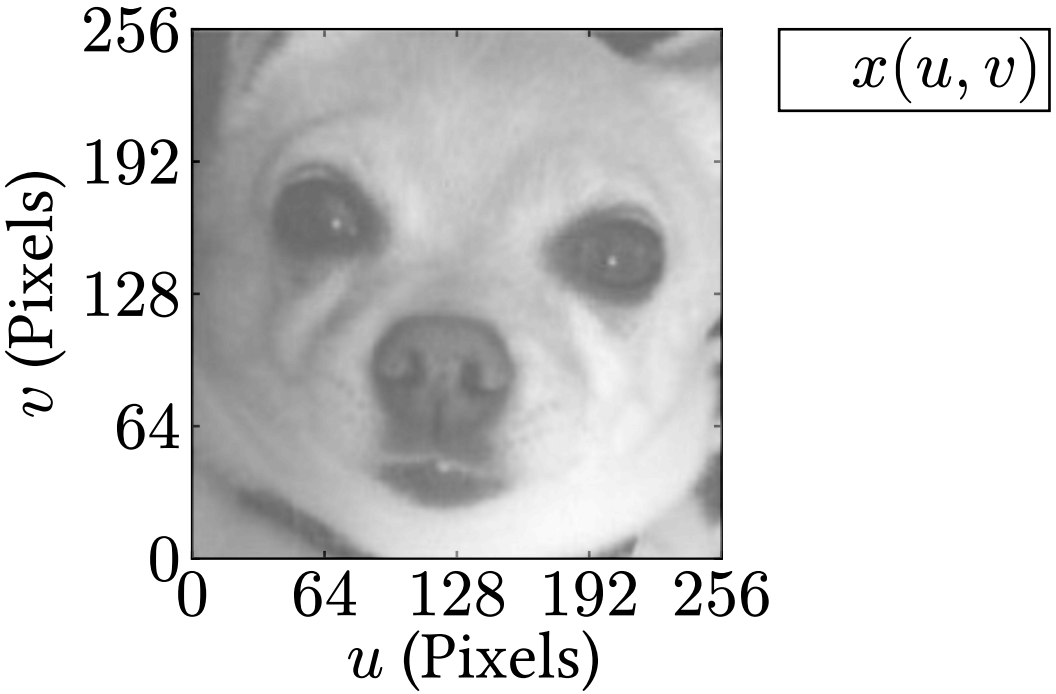


Structure: Nearby waves form syllables

Structure: Nearby syllables combine to create meaning

Review

$$x(u, v) = \text{image}$$



Structure: Repeated components (circles, symmetry, eyes, nostrils, etc)

Review

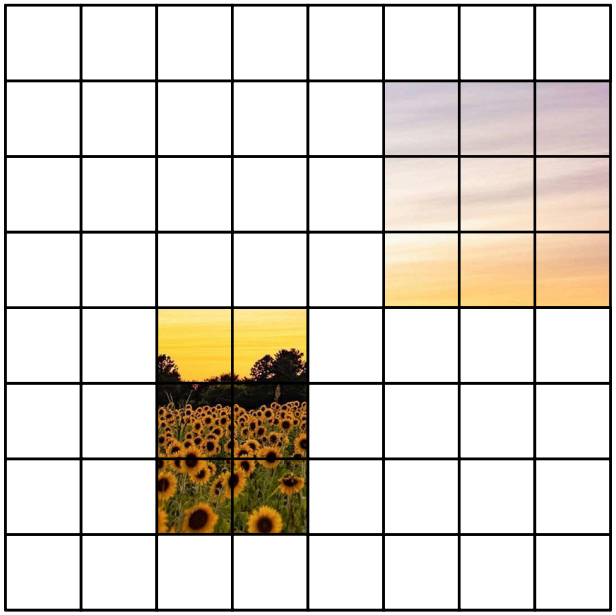
Two common properties of signals:

Locality: Information concentrated over small regions of space/time

Translation Equivariance: Shift in signal results in shift in output

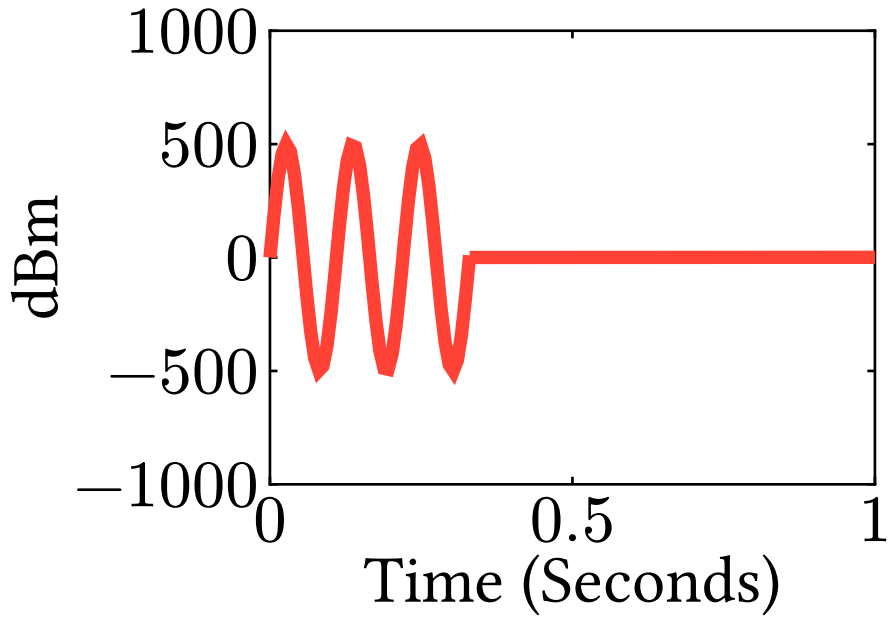
Review

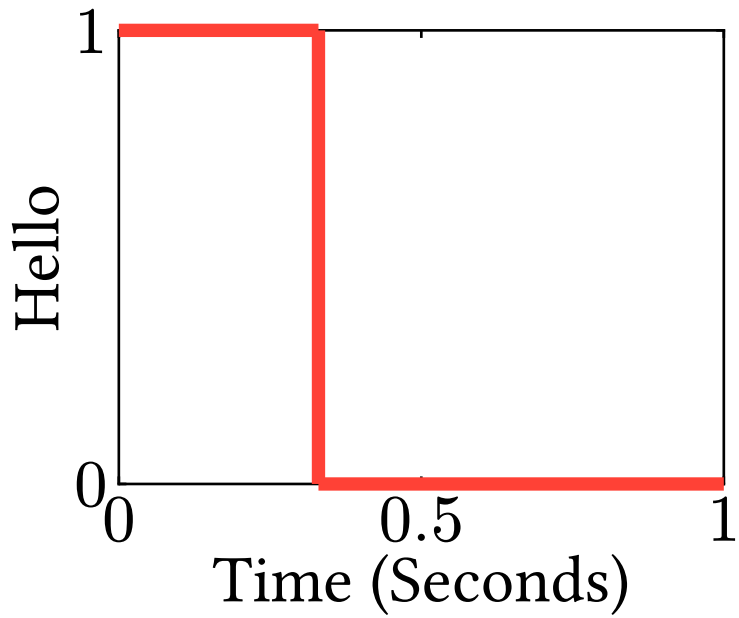
A more realistic scenario of locality and translation equivariance



Review

We use convolution to turn signals into useful signals





Convolution is translation equivariant and local

Review

Convolution is the sum of products of a signal $x(t)$ and a **filter** $g(t)$

If the t is continuous in $x(t)$

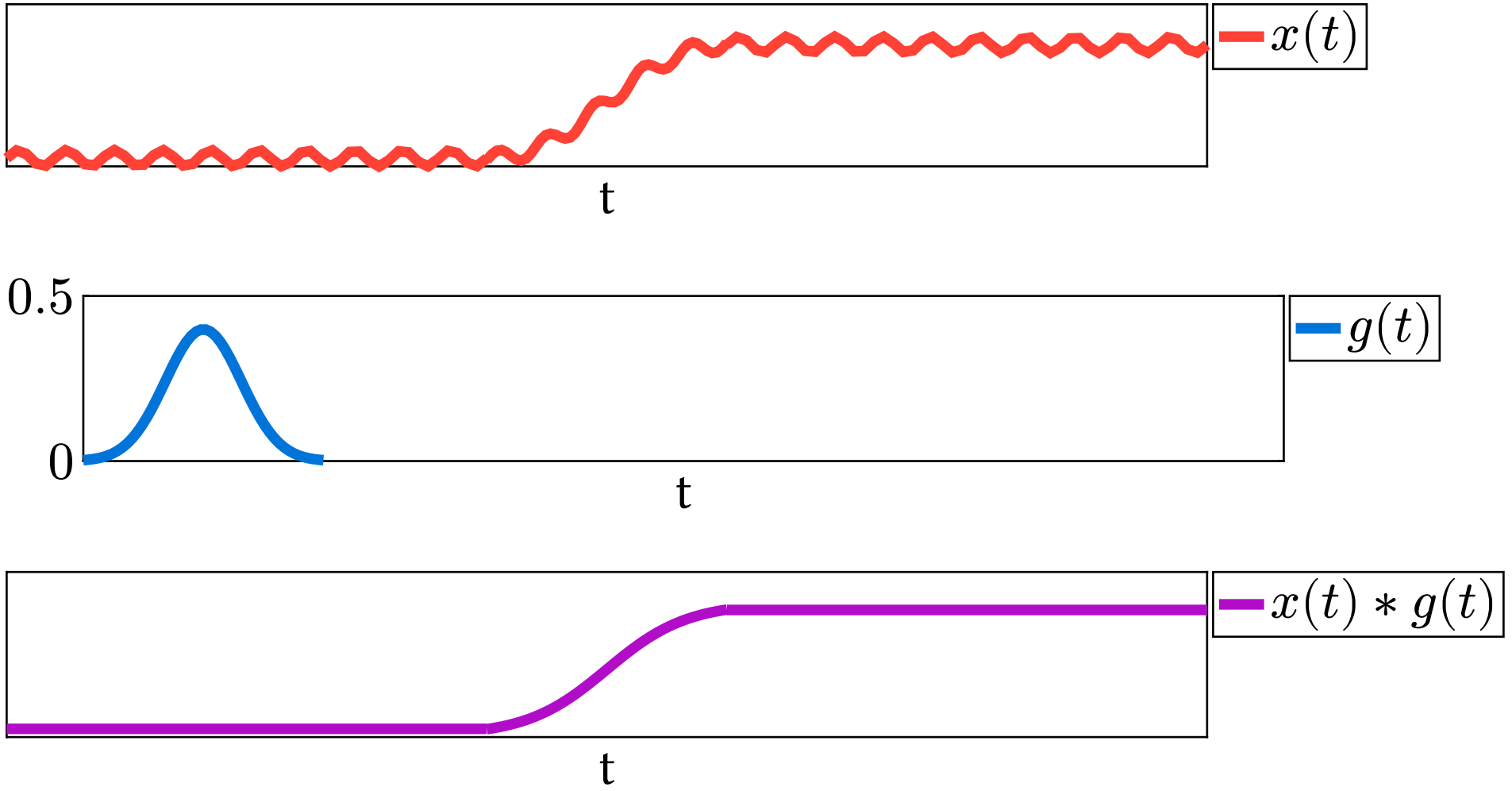
$$x(t) * g(t) = \int_{-\infty}^{\infty} x(t+\tau)g(\tau)d\tau$$

$$x(t) * g(t) = \sum_{\tau=-\infty}^{\infty} x(t+\tau)g(\tau)$$

We slide the filter $g(t)$ across the signal $x(t)$

Note: Implemented as cross-correlation, not classical convolution

Review



Review

We can write both a perceptron and convolution in vector form

$$f(x(t), \boldsymbol{\theta}) = \underbrace{\sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \\ \vdots \end{bmatrix}\right)}_{\text{Perceptron}} \qquad f(x(t), \boldsymbol{\theta}) = \underbrace{\begin{bmatrix} \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix}\right) \\ \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix}\right) \\ \vdots \end{bmatrix}}_{\text{Convolution layer}}$$

A convolution layer applies a "mini" perceptron to every few timesteps

The output size depends on the signal length

Review

If we want a single output, we should use **pooling**

$$z(t) = f(x(t), \boldsymbol{\theta}) = \left[\sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix} \right) \quad \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix} \right) \quad \dots \right]^\top$$

$$\text{SumPool}(z(t)) = \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.1) \\ x(0.2) \end{bmatrix} \right) + \sigma\left(\boldsymbol{\theta}^\top \begin{bmatrix} 1 \\ x(0.2) \\ x(0.3) \end{bmatrix} \right) + \dots$$

$$\text{MeanPool}(z(t)) = \frac{\text{SumPool}(z(t))}{T - k + 1}; \quad \text{MaxPool}(z(t)) = \max(z(t))$$

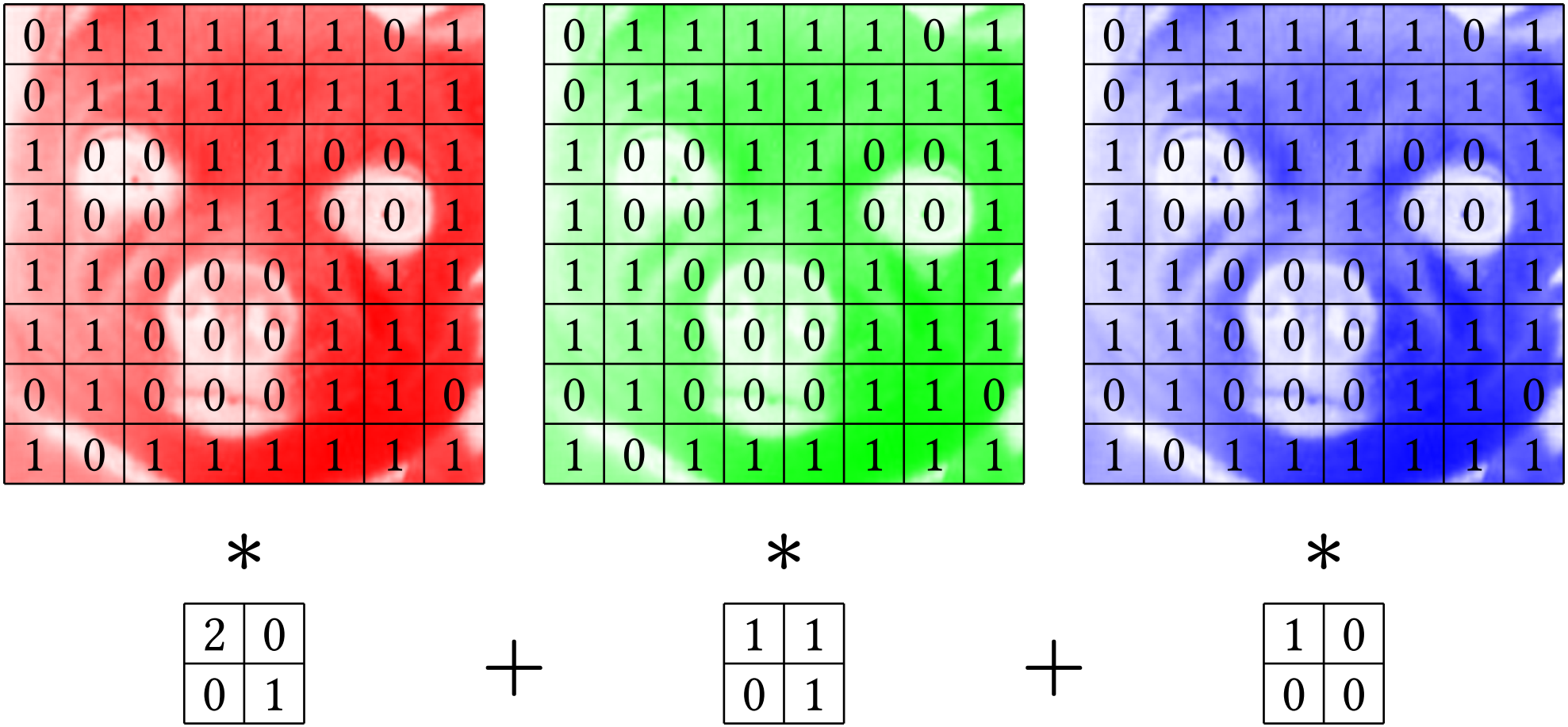
Review

Our examples considered:

- 1 dimensional variable t
- 1 dimensional output/channel $x(t)$
- 1 filter

Can expand to multiple dimensions, the idea is exactly the same

Review


0 1 1 1 1 1 0 1
0 1 1 1 1 1 1 1
1 0 0 1 1 0 0 1
1 0 0 1 1 0 0 1
1 1 0 0 0 1 1 1
1 1 0 0 0 1 1 1
0 1 0 0 0 1 1 0
1 0 1 1 1 1 1 1
0 1 1 1 1 1 0 1
0 1 1 1 1 1 1 1
1 0 0 1 1 0 0 1
1 0 0 1 1 0 0 1
1 1 0 0 0 1 1 1
1 1 0 0 0 1 1 1
0 1 0 0 0 1 1 0
1 0 1 1 1 1 1 1
0 1 1 1 1 1 0 1
0 1 1 1 1 1 1 1
1 0 0 1 1 0 0 1
1 0 0 1 1 0 0 1
1 1 0 0 0 1 1 1
1 1 0 0 0 1 1 1
0 1 0 0 0 1 1 0
1 0 1 1 1 1 1 1
*
2 0
0 1
+
*
1 1
0 1
+
*
1 0
0 0

Review

Stride allows you to "skip" cells during convolution

- Decrease the size of image without pooling

Padding adds zero pixels to the image to increase the output size

0	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1
1	0	0	1	1	0	0	1
1	0	0	1	1	0	0	1
1	1	0	0	0	1	1	1
1	1	0	0	0	1	1	1
0	1	0	0	0	1	1	0
1	0	1	1	1	1	1	1

$*$

1	0
0	1

Sequence Modeling

Sequence Modeling

We previously used convolution to model signals

Many interesting signals depend on **time**

- Stock market
- Audio transcription
- Video classification

We also call these time-varying signals **sequences**

$$x(1), x(2), ...$$

Today, we examine a new way to model time-varying signals/sequences

Sequence Modeling

Convolution: Electrical engineering approach to sequence modeling

Recurrent Models: Neuroscience approach to sequence modeling

You can use convolution or recurrent models to model sequences

- Stock market
- Audio transcription
- Video classification

So what is the difference between convolution and recurrent models?

Sequence Modeling

Convolution works over signals of any domain (time, space, graph, etc)

Recurrent neural networks only work with time

Convolution relies on locality and translation equivariance properties

Recurrent models do not assume locality or equivariance

Equivariance and locality make learning more efficient, but not all problems have this structure

- Best solution depends on your problem structure

Examine some real life time-varying signals, and see if these properties hold

Sequence Modeling

Example 1: You like dinosaurs as a child, you grow up and study dinosaurs for work

Question: Is this local?

Answer: No, two related events separated by 20 years

Sequence Modeling

Example 2: Your parent changes your diaper

Question: Translation equivariant?

Answer: No! Ok if you are a baby, different meaning if you are an adult!

Sequence Modeling

Example 3: You hear a gunshot then see runners



Question: Translation equivariant?

Answer: No! (1) gunshot, (2) see runners, enjoy the race. (1) see runners, (2) hear gunshot, you start running too!

Sequence Modeling

Problems without locality and translation equivariance are difficult to solve with convolution

For these problems, we need something else!

Can use humans as inspiration, life is a sequence/time-varying signal

How do humans experience time and process temporal data?

Can we design a neural network based on human perceptions of time?

Composite Memory

Composite Memory

How do humans process temporal data/time-varying signals?

We only perceive the present

See dog $\rightarrow$ photoreceptors fire $\rightarrow$ neurons fire in the brain

No dog $\rightarrow$ no photoreceptors fire $\rightarrow$ no neurons fire

This process only considers the present

We know there was a dog, even if we no longer see it

We can reason over time by recording information as **memories**

Humans process temporal data by storing and recalling memories

Composite Memory



John Locke (1690) believed that conciousness and identity arise from memories

If all your memories were erased, you would be a different person

Without the ability to reason over memories, we would only react to stimuli like bacteria

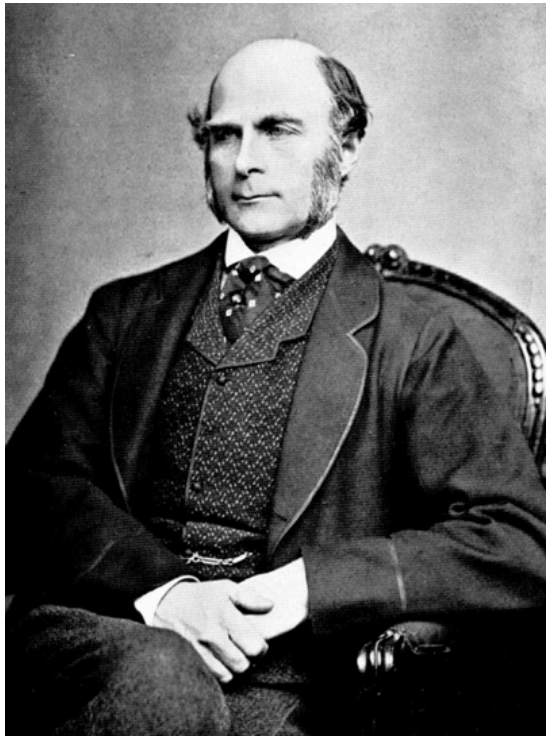
Composite Memory

So how do we model memory in humans?

Can we create a similar model for neural networks?

Composite Memory

Francis Galton (1822-1911) photo composite memory



Composite photo of members of a party



Composite Memory

Task: Find a mathematical model of composite memory

$$X : \mathbb{R}^{h \times w} \quad \text{Person you see at the party}$$

$$H : \mathbb{R}^{h \times w} \quad \text{The memories in your mind}$$

$$f : X^T \times \Theta \mapsto H$$

Composite photography/memory uses a weighted sum

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i$$

Composite Memory

Memories from the party $\boldsymbol{h}_T = \sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i$

What if we see someone at $T+1$? $\boldsymbol{h}_{T+1} = \left(\sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i \right) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T+1}$

And another new person? $\boldsymbol{h}_{T+1} = \left(\sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i \right) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T+1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T+2}$

We repeat the same process for each new face

We can rewrite f as a **recurrent function**

Linear Recurrence

Linear Recurrence

Let us rewrite composite memory as a recurrent function

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{\left(\sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i \right)}_{\boldsymbol{h}} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{\text{new}}$$

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

$$\boldsymbol{x} \in \mathbb{R}^{d_x}, \quad \boldsymbol{h} \in \mathbb{R}^{d_h}$$

Linear Recurrence

$$\boldsymbol{x} \in \mathbb{R}^{d_x}, \quad \boldsymbol{h} \in \mathbb{R}^{d_h} \qquad f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

$$\boldsymbol{h}_1 = f(\mathbf{0}, \boldsymbol{x}_1, \boldsymbol{\theta}) = \mathbf{0} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1$$

$$\boldsymbol{h}_2 = f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) = \boldsymbol{h}_1 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2$$

$$\boldsymbol{h}_3 = f(\boldsymbol{h}_2, \boldsymbol{x}_3, \boldsymbol{\theta}) = \boldsymbol{h}_2 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3$$

$$\vdots$$

$$\boldsymbol{h}_T = f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) = \boldsymbol{h}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

Linear Recurrence

Question: What is the meaning of $\boldsymbol{h}$ in humans?



Linear Recurrence

Right now, our model remembers everything

But $\boldsymbol{h}$ is a fixed size, what if T is very large?

If we keep adding and adding $\boldsymbol{x}$ into $\boldsymbol{h}$, we will run out of space

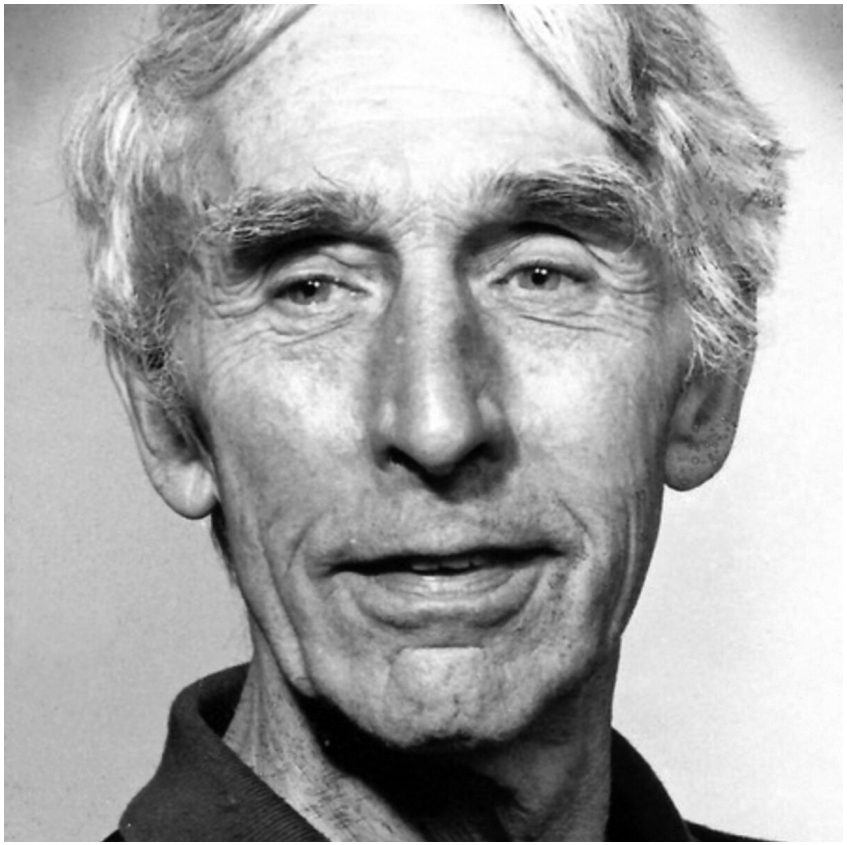
We need to create space for new memories

https://www.youtube.com/watch?v=IQ8Aak-k5Yc

Humans make space for new memories by forgetting old memories

Linear Recurrence

Murdock (1982)



$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \gamma \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}; \quad 0 < \gamma < 1$$

Key Idea: $\lim_{T \to \infty} \gamma^T = 0$

Let $\gamma = 0.9$

$$0.9 \cdot \boldsymbol{h} = 0.9\boldsymbol{h}$$

$$0.9 \cdot 0.9 \cdot \boldsymbol{h} = 0.81\boldsymbol{h}$$

$$0.9 \cdot 0.9 \cdot 0.9 \cdot \ldots \cdot \boldsymbol{h} = 0$$

Let us work out the math

Linear Recurrence

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \gamma \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}; \quad 0 < \gamma < 1$$

$$\boldsymbol{h}_T = \gamma \boldsymbol{h}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

Question: What is $\boldsymbol{h}_{T-1}$ in terms of $\boldsymbol{h}_{T-2}$?

$$\boldsymbol{h}_{T-1} = \gamma \boldsymbol{h}_{T-2} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1}$$

$$\boldsymbol{h}_T = \gamma(\gamma \boldsymbol{h}_{T-2} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1}) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

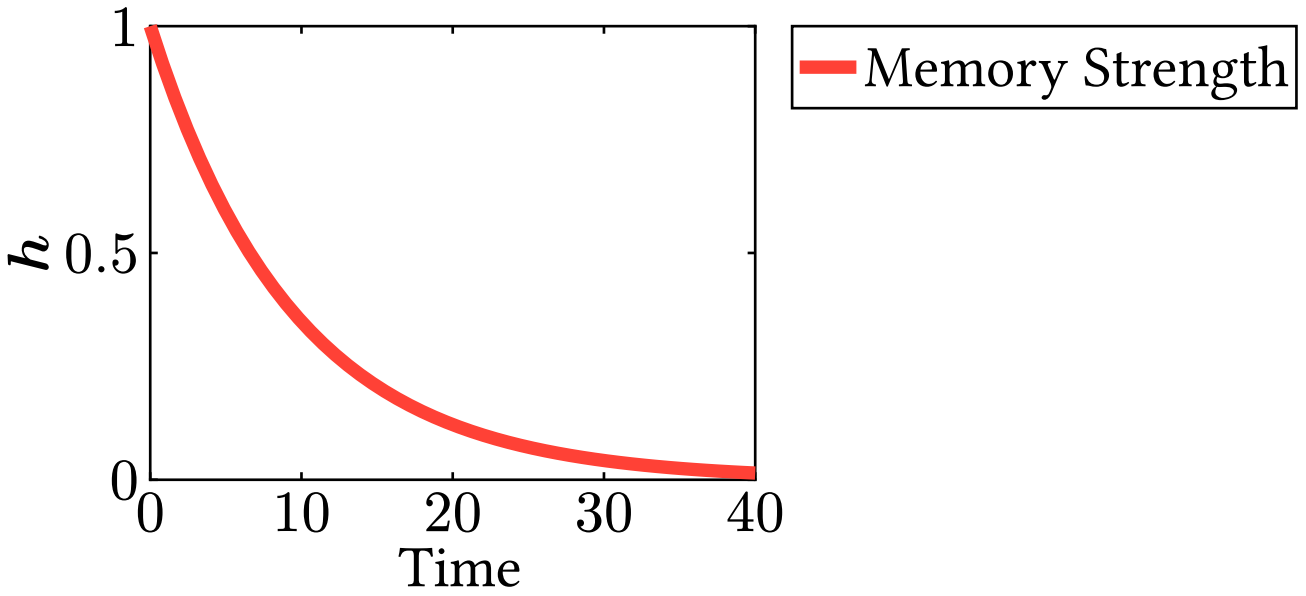
$$\boldsymbol{h}_T = \gamma(\gamma(\gamma \boldsymbol{h}_{T-3} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-2}) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1}) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

$$\boldsymbol{h}_T = \gamma^3 \boldsymbol{h}_{T-3} + \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-2} + \gamma \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

Linear Recurrence

$$\boldsymbol{h}_T = \gamma^3 \boldsymbol{h}_{T-3} + \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-2} + \gamma \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

$$\boldsymbol{h}_T = \gamma^{T-1} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1 + \gamma^{T-2} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2 + \ldots + \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-3} + \gamma \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$



Linear Recurrence

As T increases, we add new information $\boldsymbol{x}_T$

As T increases, we slowly forget old information

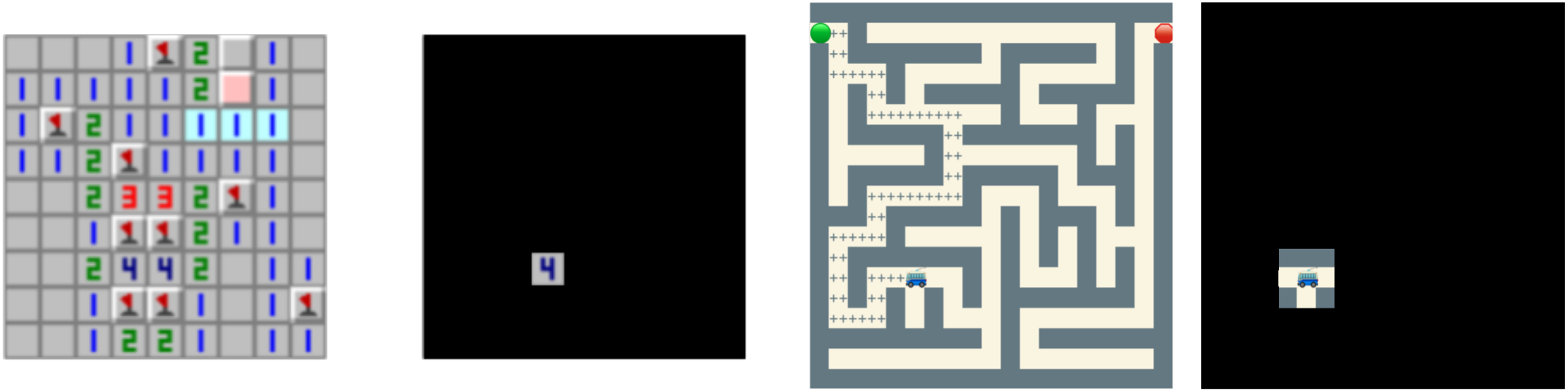
The memory decay is smooth and differentiable

We can learn the parameters $\gamma, \boldsymbol{\theta}$ using gradient descent

Linear Recurrence

Morad et al., *Reinforcement Learning with Fast and Forgetful Memory.* Neural Information Processing Systems. (2024).

$$\boldsymbol{H}_T = \boldsymbol{\gamma}\boldsymbol{H}_{T-1} + g(\boldsymbol{x}_T)$$



https://www.youtube.com/watch?v=0ey63XPB-4U&t=85s

Scans

Scans

$$\boldsymbol{h}_1 = f(\mathbf{0}, \boldsymbol{x}_1, \boldsymbol{\theta}) = \gamma \mathbf{0} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1$$

$$\boldsymbol{h}_2 = f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) = \gamma \boldsymbol{h}_1 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2$$

$$\boldsymbol{h}_3 = f(\boldsymbol{h}_2, \boldsymbol{x}_3, \boldsymbol{\theta}) = \gamma \boldsymbol{h}_2 + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3$$

$$\vdots$$

$$\boldsymbol{h}_T = f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) = \gamma \boldsymbol{h}_{T-1} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_T$$

How do we compute $\boldsymbol{h}_1, \boldsymbol{h}_2, ..., \boldsymbol{h}_T$ on a computer?

We use an algebraic operation called a **scan**

Scans

Our function f is just defined for a single X

$$f : H \times X \times \Theta \mapsto H$$

But we need to process a sequence X^T!

To extend f to sequences, we scan f over the inputs

$$\text{scan}(f) : H \times X^T \times \Theta \mapsto H^T$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \vdots \\ \boldsymbol{h}_T \end{bmatrix}$$

Scans

$$f : H \times X \times \Theta \mapsto H, \quad \text{scan}(f) : H \times X^T \times \Theta \mapsto H^T$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_T \end{bmatrix}$$

$$\text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} f(\boldsymbol{h}_0, \boldsymbol{x}_1, \boldsymbol{\theta}) \\ f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) \\ \vdots \\ f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} f(\boldsymbol{h}_0, x_1) \\ f(f(\boldsymbol{h}_0, \boldsymbol{x}_1), \boldsymbol{x}_2) \\ \vdots \\ f(...f(\boldsymbol{h}_0, \boldsymbol{x}_1)..., \boldsymbol{x}_T) \end{bmatrix}$$

Scans

`torch` does NOT have built-in scans.

We will write our own scan

```
def scan(f, h, xs):
    # h shape is (d_h,)
    # xs shape is (T, d_x)
    hs = []
    for x in xs:
        h = f(h, x, theta)
        hs.append(h)
    # output shape is (T, d_h)
    return torch.stack(hs)
```

Scans

```
import torch
T, d_x, d_h = 10, 2, 4

xs, h0 = torch.ones((T, d_x)), torch.zeros((d_h,))
theta = (torch.ones((d_h,)), torch.ones((d_x, d_h)))

def f(h, x):
    b, W = theta
    result = h + (W.T @ x + b)
    return result # h

hs = scan(f, h0, xs)
```

Scans

Many deep learning courses do not teach scans

I teach scans because they are an important part of future LLMs

Companies are experimenting with LLMs using **associative scans**

- Qwen3-Next, Google RecurrentGemma, Griffin

Standard LLM: ~1M words

Associative LLM: 100M words

An associative scan is a very fast scan we use when f obeys the associative property $f(f(h_1, x_2), x_3) = f(h_1, f(x_2, x_3))$

Question: Does $f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \gamma \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$ obey the associative property?

Answer: Yes, linear operations obey the associative property

Output Modeling

Output Modeling

We are almost done defining recurrent models

There is one more step we must consider, **memory recall**

$\boldsymbol{h}$ stores all memories, but humans only access a few memories at once

Example: I ask you your favorite ice cream flavor

You recall previous times you ate ice cream, but not your phone number

We will model this recall of memories using a function g

Output Modeling

Let g define our memory recall function

$$g : H \times X \times \Theta \mapsto Y$$

g searches your memories h using the input x, to produce output y

$\boldsymbol{x}$: "What is your favorite ice cream flavor?"

$\boldsymbol{h}$: Everything you remember (hometown, birthday, etc)

g : Searches your memories for ice cream information, and responds "chocolate"

Now, we will combine f and g

Output Modeling

Step 1: Perform scan to find recurrent states

$$\begin{bmatrix} \boldsymbol{h}_1 \\ \vdots \\ \boldsymbol{h}_T \end{bmatrix} = \text{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}_f \right)$$

Step 2: Perform recall on recurrent states

$$\begin{bmatrix} \hat{\boldsymbol{y}}_1 \\ \vdots \\ \hat{\boldsymbol{y}}_T \end{bmatrix} = \begin{bmatrix} g(\boldsymbol{h}_1, \boldsymbol{x}_1, \boldsymbol{\theta}_g) \\ \vdots \\ g(\boldsymbol{h}_T, \boldsymbol{x}_T, \boldsymbol{\theta}_g) \end{bmatrix}$$

Questions? This is on the homework

Output Modeling

To summarize, we defined:

- Recurrent function f
- Scanned recurrence $\text{scan}(f)$
- Output function g

To run our model:

- Execute $\text{scan}(f)$ over inputs to make recurrent states
- Execute g over recurrent states to make outputs

Recurrent Loss Functions

Recurrent Loss Functions

Let us examine some example tasks:

- Clock
- Explaining a video

Recurrent Loss Functions

Task: Robot navigation

Input is robot velocity, output is robot position

$$X \in \mathbb{R}^2, \quad Y \in \mathbb{R}^2$$

Recurrent Loss Functions

Example input sequence:

$$\begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix} = \begin{bmatrix} \dot{u}_1 & \dot{v}_1 \\ \vdots & \vdots \\ \dot{u}_T & \dot{v}_T \end{bmatrix}$$

Desired output sequence

$$\begin{bmatrix} \boldsymbol{y}_1 \\ \vdots \\ \boldsymbol{y}_T \end{bmatrix} = \begin{bmatrix} u_1 & v_1 \\ \vdots & \vdots \\ u_T & v_T \end{bmatrix}$$

We have a corresponding label $\boldsymbol{y}$ for each input x

This is one datapoint (single datapoint has T elements)

- Dataset contains $n \times T$ positions and velocities

Recurrent Loss Functions

Regression task, can use square error

First, scan f over the inputs to find h

$$\begin{bmatrix} \boldsymbol{h}_{[i],1} \\ \vdots \\ \boldsymbol{h}_{[i],T} \end{bmatrix} = \text{scan}(f) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_{[i],1} \\ \vdots \\ \boldsymbol{x}_{[i],T} \end{bmatrix}, \boldsymbol{\theta}_f \right)$$

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{T} \sum_{k=1}^{d_y} \left[g\left(\boldsymbol{h}_{[i],j}, \boldsymbol{x}_{[i],j}, \boldsymbol{\theta}_g \right)_k - y_{[i],j,k} \right]^2$$

Onto the next task

Recurrent Loss Functions

Task: Watch a video, then explain it

$$X \in \mathbb{Z}^{3\times 32\times 32}, \quad Y \in \{\text{comedy show}, \text{action movie}, \ldots\}$$

Example input sequence:

$$\begin{bmatrix} \boldsymbol{X}_1 \\ \boldsymbol{X}_2 \\ \vdots \\ \boldsymbol{X}_T \end{bmatrix}$$

Example output: "dancing dog"

Unlike before, we have many inputs but just one output!

Recurrent Loss Functions

We will use the classification loss

We scan f over the sequence, then compute g for the final timestep

$$\begin{bmatrix} \boldsymbol{h}_{[i],1} \\ \vdots \\ \boldsymbol{h}_{[i],T} \end{bmatrix} = \text{scan}(f) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_{[i],1} \\ \vdots \\ \boldsymbol{x}_{[i],T} \end{bmatrix}, \boldsymbol{\theta}_f \right)$$

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_y} y_{[i],j} \log g\Big(\boldsymbol{h}_{[i],T}, \boldsymbol{x}_{[i],T}, \boldsymbol{\theta}_g\Big)_j$$

We only care about the $\boldsymbol{h}_T$

Recurrent Loss Functions

To summarize, we use standard losses for recurrent loss functions

$$\sum_{i=1}^{n} \sum_{j=1}^{T} \sum_{k=1}^{d_y} \dots$$

Just be careful – we often sum over an additional axis

Backpropagation Through Time

Backpropagation Through Time

1. We created the model
2. We found the loss function
3. Now we need to find the parameters!

Just like other models, we train recurrent models using gradient descent

- State $\boldsymbol{h}$ updates over time, we must **backpropagate through time**

Step 1: Compute gradient

Step 2: Update parameters using gradient

How do we compute gradients for recurrent functions?

Backpropagation Through Time

First, compute gradient of f

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \gamma \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

Question: What is $(\nabla_{\boldsymbol{\theta}} f)(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta})$?

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \overline{\boldsymbol{x}}$$

Too easy, now let us find the gradient of $\mathrm{scan}(f)$

Backpropagation Through Time

$$\operatorname{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix}\boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} f(\boldsymbol{h}_0, \boldsymbol{x}_1, \boldsymbol{\theta}) \\ f(\boldsymbol{h}_1, \boldsymbol{x}_2, \boldsymbol{\theta}) \\ \vdots \\ f(\boldsymbol{h}_{T-1}, \boldsymbol{x}_T, \boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} f(\boldsymbol{h}_0, \boldsymbol{x}_1) \\ f(f(\boldsymbol{h}_0, \boldsymbol{x}_1), \boldsymbol{x}_2) \\ \vdots \\ f(...f(\boldsymbol{h}_0, \boldsymbol{x}_1)..., \boldsymbol{x}_T) \end{bmatrix}$$

Question: What is $\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)$?

Hint: Chain rule

$$(\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f))\left(\boldsymbol{h}_0, \begin{bmatrix}\boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} (\nabla_{\boldsymbol{\theta}} f)(\boldsymbol{h}_0, \boldsymbol{x}_1) \\ (\nabla_{\boldsymbol{\theta}} f)(f(\boldsymbol{h}_0, \boldsymbol{x}_1), \boldsymbol{x}_2)(\nabla f)(\boldsymbol{h}_0, \boldsymbol{x}_1) \\ \vdots \\ (\nabla_{\boldsymbol{\theta}} f)(...f(\boldsymbol{h}_0, \boldsymbol{x}_1)..., \boldsymbol{x}_T)...(\nabla f)(\boldsymbol{h}_0, \boldsymbol{x}_1) \end{bmatrix}$$

Backpropagation Through Time

$$(\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta} \right) = \begin{bmatrix} \nabla_{\boldsymbol{\theta}} f \\ (\nabla_{\boldsymbol{\theta}} f)\ (\nabla f) \\ (\nabla_{\boldsymbol{\theta}} f)\ (\nabla f)\ (\nabla f) \\ \vdots \end{bmatrix}$$

Question: Any issues with this?

Hint: What if $\nabla_{\boldsymbol{\theta}} f$ is $\ll 1$ or $\gg 1$? $(\nabla_{\boldsymbol{\theta}} f)^T$

$$\lim_{t \to \infty} 0.99^T = 0 \qquad \lim_{t \to \infty} 1.01^T = \infty$$

Read: *Untersuchungen zu dynamischen neuronalen Netzen* or *Learning Long-Term Dependencies with Gradient Descent is Difficult*

Recurrent Neural Networks

Recurrent Neural Networks

Until now, f was a linear function

If we make f a neural network, then we have a **recurrent neural network** (RNN)

Recurrent Neural Networks

Here is our linear recurrence

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

Question: How do we make it a neural network?

$$\sigma\left(\boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right) \qquad \boldsymbol{h} + \sigma\left(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right) \qquad \sigma(\boldsymbol{h}) + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}$$

All work, and all are recurrent neural networks!

- In practice, we often use $\sigma\left(\boldsymbol{h} + \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}\right)$ form

Recurrent Neural Networks

The simplest common recurrent neural network is the **Elman Network**

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{h} + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}})$$

$$g(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \boldsymbol{\theta}_3^\top \boldsymbol{h}$$

Apply sigmoid activation to $\boldsymbol{h}$ to keep the recurrent state bounded

- Otherwise it can become very large over long sequences

Question: Anything missing from our linear model?

Answer: Forgetting!

Recurrent Neural Networks

Add forgetting

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \boldsymbol{h} \odot f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}}\right)$$

$$f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_2^\top \boldsymbol{h}\right)$$

Question: σ is sigmoid. What is co-domain of f_{forget}?

Answer: $[0, 1]$

- When $f_{\text{forget}} = 1$, we forget nothing!
- When $f_{\text{forget}} = 0$, we forget everything!
- When $0 < f_{\text{forget}} < 1$, we forget some things!

Neural network learns which memories to forget with gradient descent

Recurrent Neural Networks

Minimal gated unit (MGU) is a modern RNN

MGU defines two helper functions

$$f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_2^\top \boldsymbol{h}\right)$$

$$f_2(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{\sigma\left(\boldsymbol{\theta}_3^\top \overline{\boldsymbol{x}} + \boldsymbol{\theta}_4^\top f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) \odot \boldsymbol{h}\right)}_{\text{Make new memories}}$$

$$f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) \odot \boldsymbol{h}}_{\text{Forget old memories}} + \underbrace{\left(1 - f_{\text{forget}}(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta})\right)}_{\text{Replace}} \odot \underbrace{f_2(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta})}_{\text{New memories}}$$

Only forget when we have new information to remember

Recurrent Neural Networks

There are even more complicated models

- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

LSTM has 6 different functions! Too complicated to review

MGU is simpler and performs similarly to LSTM and GRU

Recurrent Neural Networks

Recall the gradient for the linear recurrence

$$\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)\left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \begin{bmatrix} \nabla_{\boldsymbol{\theta}} f \\ (\nabla_{\boldsymbol{\theta}} f)(\nabla f) \\ (\nabla_{\boldsymbol{\theta}} f)(\nabla f)(\nabla f) \\ \vdots \end{bmatrix}$$

Elman network f: $\quad f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{h} + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}})$

Question: What is the gradient for $\operatorname{scan}(f)$ of Elman network?

Recurrent Neural Networks

Elman network f: $f(\boldsymbol{h}, \boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_1^\top \boldsymbol{h} + \boldsymbol{\theta}_2^\top \overline{\boldsymbol{x}})$

$$(\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta} \right) = \begin{bmatrix} \nabla_{\boldsymbol{\theta}} f \\ (\nabla_{\boldsymbol{\theta}} f)(\nabla_{\boldsymbol{\theta}} f) \\ (\nabla_{\boldsymbol{\theta}} f)(\nabla_{\boldsymbol{\theta}} f)(\nabla_{\boldsymbol{\theta}} f) \\ \vdots \end{bmatrix}$$

$$(\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta} \right) = \begin{bmatrix} \nabla \sigma \\ (\nabla \sigma)(\nabla \sigma) \\ (\nabla \sigma)(\nabla \sigma)(\nabla \sigma) \\ \vdots \end{bmatrix}$$

Recurrent Neural Networks

$$(\nabla_{\boldsymbol{\theta}} \operatorname{scan}(f)) \left(\boldsymbol{h}_0, \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta} \right) = \begin{bmatrix} \nabla\sigma \\ (\nabla\sigma)(\nabla\sigma) \\ (\nabla\sigma)(\nabla\sigma)(\nabla\sigma) \\ \vdots \end{bmatrix}$$

Question: What's the problem?

Answer: Vanishing gradient $(\nabla\sigma) \cdot (\nabla\sigma) \cdot \ldots = 0$

Recurrent Neural Networks

All modern RNNs suffer from either exploding or vanishing gradient

- Very active area of research
- Could replace attention and yield much more efficient LLMs

Usually, we choose vanishing gradient

- Vanishing gradient prevents learning interactions over long T
- Exploding gradient produces `NaN` and crashes training

Other works select eigenvalues near 1 to bound gradients

$$\nabla \sigma(\boldsymbol{\theta}^\top \boldsymbol{x}) = \lambda \boldsymbol{x}; \quad \lambda \approx 1$$

But this restricts the power of the neural network

Coding

Coding

Jax RNN https://colab.research.google.com/drive/18DgEVjJj6JDzqcD595PaCuyucNFJIO-M?usp=sharing

Homework https://colab.research.google.com/drive/1vqIlVVHtaNt8i-XwXRye6eggv_cntIHh?usp=sharing